# BdOI Monthly Contest Round1 Solutions: Problem Summand

## Problem Information

Problem Author: Jahin Ahnaf

Problem Preparation: Jahin Ahnaf

Editorial: Jahin Ahnaf,Sazid Hasan

## Subtasks hinting key observations

### Preliminary Observation:

The problem asks us to find the minimum number $m$ of non-negative integers $a_1, a_2, \ldots, a_m$ such that:

$$n = a_1^x + a_2^x + \cdots + a_m^x$$

Since $1^x = 1$ is true for any positive integer $x$, a valid representation always exists for any $n \geq 1$ by taking $m = n$ copies of 1. Therefore, an impossible output $(-1 - 1)$ is never triggered for valid positive inputs.

### Subtask 1: $x = 1$

When the exponent is 1, the equation simplifies to $n = a_1 + a_2 + \cdots + a_m$. Therefore, We can say $m = 1$ always for this case. We can achieve $m = 1$ by setting $a_1 = n$. Then can just print 1 and $a_1$. It will give us full score in this subtask.

### Subtask 2: $2^x > n$

In this subtask we can use our preliminary observation. As defined in the subtask $2^x > n$. Therefore, no integer greater than or equal to 2 can be used as a summand. So we can use this property for any $x^{th}$ power

$$1^x = 1$$

. So, the only way to form the sum $n$ is by using $n$ copies of 1. So the answer is $m = n$. The sequence will be $[1, 1, \ldots, 1]$ which is $n$ copies of 1. This will give us the full score in this subtask.

## Subtask 3: $m = 2$

This subtask gives us an important distinction. The answer $m$ is always 2. If we submit $m - 1$ it will reward us free partial points. For the sequence we need to test if a 2-summand partition exists. For this we can iterate from 1 upto $\lfloor \sqrt[x]{n} \rfloor$. Let's say a possible candidate is $a_1$. For each possible candidate $a_1$, we compute the remainder $rem = n - a_1^x$ and check if $rem$ is a perfect $x^{th}$ power by taking $a_2 = \lfloor \sqrt[x]{rem} \rfloor$ and verifying if $a_1^x + a_2^x = n$.

## Subtask 4 & 5:

For smaller values of $n$, this problem can be modeled as the classic coin change problem in dynamic programming. Let's define a Dynamic Programming (DP) table:

$$\text{dp}[i] = \text{minimum number of } x^{th} \text{ powers needed to sum to } i$$

In base case $\text{dp}[0] = 0$, and $\text{dp}[i] = \infty$ for all $i > 0$. In simple terms this means we need infinite amount of summands(possibly $n$) to construct our sum. However, we update the dp state if we find a better solution. For each state $i$ from 1 to $n$, we iterate over all valid bases $j \geq 1$ such that $j^x \leq i$:

$$\text{dp}[i] = \min_{1 \leq j^x \leq i} (\text{dp}[i - j^x] + 1)$$

This will yield us the value of $m$ which will reward us in partial points. In order to construct the sequence, we maintain a parent[i] array that records the optimal base $j$ used to transition into state $i$. By backtracking from $n$ to 0, we collect the summands and sort them in non-decreasing order.

## Subtask 6 & 7(Full Solution)

If we compute the dp table from scratch for every test case, it will result in Time Limit Exceeded (TLE). So, it's not possible to compute dp table in every test cases as test cases upto be $T \leq 1000$. Computing dp every time for a large number of test cases will reward us in TLE. However, one noticeable thing is x is very small ($1 \leq x \leq 10$). There are only 9 non-trivial exponents ($x \in [2, 10]$) we need to worry about. We can precompute the dp tables globally once up to $N_{\max} = 10^5$ before answering any queries! We can make a slight adjustment to our previous dp table. Let, $\text{dp}[x][i]$ be the minimum number of $x^{th}$ powers needed to sum to the integer $i$. For each exponent $x \in [2, 10]$ the base case is $\text{dp}[x][0] = 0$ (a sum of 0 requires zero summands) and $\text{dp}[x][i] = \infty$ for all $1 \leq i \leq N_{\max}$. For each $x \in [2, 10]$ and for each target sum $i$ from 1 to $N_{\max}$ we compute the following dp table:

$$\text{dp}[x][i] = \min_{1 \leq j^x \leq i} (\text{dp}[x][i - j^x] + 1)$$

This will yield us the minimum number of summands, which will reward us in partial points. In order to construct the sequence, we maintain a $\text{parent}[x][i] = j$ array that records the optimal base $j$ used to transition into state $i$. By backtracking from $n$ to 0, we collect the summands and sort them in non-decreasing order.